

# Formation Python pour l'analyse de données

Fiacre Bandaogo

## Table des matières

|                                                                       |          |
|-----------------------------------------------------------------------|----------|
| <b>Formation Python pour l'analyse de données</b>                     | <b>1</b> |
| Programme détaillé — 20 séances de 2h30                               | 1        |
| 1. Cadrage préalable                                                  | 1        |
| 2. Vue d'ensemble du parcours                                         | 2        |
| 3. Module 0 — Installation (à faire avant la séance 1)                | 3        |
| 4. Module 1 — Fondamentaux du langage (séances 1 à 5)                 | 5        |
| 5. Module 2 — NumPy (séances 6 et 7)                                  | 8        |
| 6. Module 3 — pandas (séances 8 à 12)                                 | 9        |
| 7. Module 4 — Import et nettoyage (séances 13 et 14)                  | 11       |
| 8. Module 5 — Visualisation (séances 15 à 17)                         | 12       |
| 9. Module 6 — Statistiques et EDA (séances 18 et 19)                  | 14       |
| 10. Séance 20 — Restitution du projet final                           | 15       |
| 11. Fil transversal — Code répliquable et dossier de travail organisé | 16       |
| 12. Recommandations opérationnelles                                   | 19       |
| 13. Récapitulatif du volume horaire                                   | 20       |

## Formation Python pour l'analyse de données

### Programme détaillé — 20 séances de 2h30

---

#### 1. Cadrage préalable

##### 1.1 Public visé

Participants sans aucune connaissance de Python, ni de programmation en général. Aucun prérequis mathématique au-delà du niveau lycée. La formation suppose une aisance basique avec un ordinateur (fichiers, dossiers, tableur).

##### 1.2 Objectif

**Objectif de sortie : autonomie opérationnelle.** À l'issue de la formation, un participant sait prendre un jeu de données brut (CSV, Excel, base SQL), le nettoyer, l'explorer, en tirer des statistiques et des visualisations pertinentes, et restituer ses conclusions — sans assistance, sur un cas qu'il n'a jamais vu. C'est le niveau « data analyst junior opérationnel ». À la clôture (section 10), une feuille de route est également prévue pour ceux qui souhaitent approfondir leur maîtrise et acquérir de l'expertise.

##### 1.3 Format

| Paramètre                    | Valeur                                       |
|------------------------------|----------------------------------------------|
| Durée totale                 | 20 séances × 2h30 = <b>50 heures</b>         |
| Rythme                       | 1 séance hebdomadaire                        |
| Étalement                    | ~5 mois (prévoir 22 semaines avec les aléas) |
| Travail personnel            | 1h à 1h30 entre chaque séance                |
| Taille de groupe recommandée | 8 à 12 participants                          |
| Environnement                | Anaconda + JupyterLab (installation locale)  |

### 1.4 Principe pédagogique directeur

**Ratio 30/70.** Maximum 45 minutes d'exposé par séance de 2h30. Le reste est du clavier sous les doigts des participants. Une notion non pratiquée dans les 10 minutes qui suivent son explication est une notion perdue.

Chaque séance suit la même structure, ce qui crée un rythme rassurant :

| Temps       | Phase          | Contenu                                                        |
|-------------|----------------|----------------------------------------------------------------|
| 0-15 min    | <b>Réveil</b>  | Correction du travail personnel, questions                     |
| 15-45 min   | <b>Exposé</b>  | Nouvelle notion, démonstration live commentée                  |
| 45-75 min   | <b>Guidé</b>   | Les participants reproduisent, le formateur circule            |
| 75-90 min   | <b>Pause</b>   | —                                                              |
| 90-135 min  | <b>Atelier</b> | Exercices en autonomie ou binômes, difficulté croissante       |
| 135-150 min | <b>Clôture</b> | Synthèse, erreurs fréquentes du jour, annonce du travail perso |

## 2. Vue d'ensemble du parcours

| Module   | Séances  | Thème                   | Compétence acquise                   |
|----------|----------|-------------------------|--------------------------------------|
| <b>0</b> | Avant S1 | Installation            | Environnement fonctionnel            |
| <b>1</b> | 1-5      | Fondamentaux du langage | Écrire un programme Python simple    |
| <b>2</b> | 6-7      | NumPy                   | Calcul numérique vectorisé           |
| <b>3</b> | 8-12     | pandas                  | Manipuler des données tabulaires     |
| <b>4</b> | 13-14    | Import & nettoyage      | Traiter des données réelles, sales   |
| <b>5</b> | 15-17    | Visualisation           | Produire des graphiques exploitables |
| <b>6</b> | 18-19    | Statistiques & EDA      | Analyser et interpréter              |
| <b>7</b> | 20       | Restitution projet      | Mener une analyse de bout en bout    |

### Fil rouge

Un **projet fil rouge unique** traverse toute la formation, introduit dès la séance 6 et enrichi à chaque module. Choisissez un jeu de données lié au métier des participants — c'est le levier d'engagement le plus puissant

dont vous disposez. Quelques pistes selon le contexte :

- Données RH (effectifs, turnover, rémunérations anonymisées)
- Données commerciales (ventes, clients, produits)
- Données publiques nationales (démographie, santé, économie)
- Données open data sectorielles

À défaut, un jeu de données grand public bien choisi fonctionne (ventes e-commerce, données de vols, données immobilières). Critères : au moins 5 000 lignes, 8 à 15 colonnes, un mélange de numérique / catégoriel / dates, et **des défauts réels** (valeurs manquantes, doublons, incohérences). Un jeu trop propre ne prépare à rien.

---

### 3. Module 0 — Installation (à faire avant la séance 1)

**Documents prêts à envoyer** (dans ce même dossier) : - `Guide_installation_Anaconda.pdf` — 7 pages, destiné aux participants - `formation_python.zip` — dossier de travail + scripts, à joindre au même envoi

À envoyer aux participants **10 jours avant** le démarrage, avec une permanence d'assistance optionnelle.

#### Contenu du guide

1. Télécharger Anaconda Distribution
2. Installation : accepter les options par défaut, **ne pas cocher** « Add to PATH » sous Windows
3. Décompresser `formation_python.zip` où ils veulent sur leur ordinateur
4. **Double-cliquer sur `lancer_jupyterlab.bat`** (ou `.command` sur Mac) — installe puis ouvre JupyterLab, 20 minutes la première fois — et exécuter le notebook de test
5. Envoyer une capture d'écran du résultat au formateur

Le guide comporte également une section de dépannage couvrant les neuf problèmes les plus fréquents, ainsi qu'une check-list finale à cocher.

#### Le dossier `formation_python.zip`

Aucune commande à taper, aucun chemin à saisir, aucun terminal à ouvrir : **tout passe par des double-clics**. C'est ce qui fait la différence avec un public débutant — la saisie de commandes et de chemins est, de loin, la première cause d'échec avant une séance 1.

| Fichier                                                    | Rôle                                                                                                                                     |
|------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| <code>lancer_jupyterlab.bat</code> / <code>.command</code> | <b>Le seul fichier que les participants utilisent.</b><br>Installe au premier lancement, met à jour si nécessaire, puis ouvre JupyterLab |
| <code>installer.bat</code> / <code>.command</code>         | Appelé automatiquement par le lanceur. Utilisable seul pour réparer                                                                      |
| <code>diagnostic.bat</code> / <code>.command</code>        | Diagnostic complet, ne modifie rien, produit <code>diagnostic.txt</code>                                                                 |
| <code>environment.yml</code>                               | Source unique de vérité pour les paquets                                                                                                 |

L'installateur crée aussi l'arborescence de travail (`seances/`, `donnees/brut/`, `donnees/propre/`, `resultats/`, `projet_final/`), ce qui supprime toute possibilité d'erreur de nommage à la création.

**L'environnement se met à jour tout seul.** La sentinelle enregistre l'empreinte d'`environment.yml`. Au lancement, le script compare : si le fichier a changé, il déclenche la mise à jour avant d'ouvrir JupyterLab. Concrètement, le jour où vous ajoutez un paquet, vous redistribuez `environment.yml` et **vous n'avez personne à prévenir**. En cas de doute — outil de hachage absent, sentinelle antérieure à cette version — rien n'est forcé : JupyterLab s'ouvre normalement.

Quatre garde-fous sont intégrés, repris d'un dispositif déjà éprouvé en conditions réelles :

- **L'environnement vit hors du dossier de travail**, dans `%LOCALAPPDATA%` (Windows) ou `~/.local/share` (macOS). Un environnement conda contient des dizaines de milliers de petits fichiers : posé dans un dossier OneDrive ou Drive, il est corrompu ou ralenti par la synchronisation. Il est reconstituable en 10 minutes, donc jamais à sauvegarder.
- **Le lanceur active conda avant de démarrer JupyterLab**. Sans cela, les noyaux plantent dès `import numpy` avec le code `0xC06D007F` — les DLL natives sont introuvables. Panne silencieuse et totalement incompréhensible pour un débutant.
- **L'installateur enregistre un noyau nommé « Python 3 (formation) »**. Cela évite qu'un ancien noyau `python3` présent sur la machine ne prenne le dessus et produise des `ModuleNotFoundError` alors que tout est correctement installé.
- **Une sentinelle `.install_complete`** empêche de lancer JupyterLab sur une installation interrompue en cours de route.

**Les avertissements non bloquants remontent.** Un noyau concurrent détecté, un enregistrement de noyau échoué — autant de choses qui n'empêchent pas l'installation d'aboutir mais qui causeront un `ModuleNotFoundError` trois semaines plus tard. Ils sont regroupés dans `avertissements.txt` et le lanceur marque un arrêt dessus avant d'ouvrir JupyterLab, pour qu'ils soient lus.

**Repartir de zéro.** Si un environnement est cassé au point que la mise à jour ne le répare pas, supprimez le dossier `%LOCALAPPDATA%\formation_python` (Windows) ou `~/.local/share/formation_python` (macOS), puis relancez `lancer_jupyterlab`. Rien d'autre n'est à toucher : le dossier de travail du participant n'est pas concerné.

**En cas de blocage, le participant n'a rien à décrire.** Il envoie `installation.log` ou `diagnostic.txt`, qui contiennent la cause exacte. C'est ce qui vous évitera une série d'échanges du type « ça ne marche pas » la veille de la séance 1.

### Le fichier `environment.yml`

Il couvre les 20 séances : **aucun paquet ne devra être installé en cours de formation.**

| Choix                                                   | Motif                                                                                                                                 |
|---------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Canal <code>conda-forge</code> + <code>nodefauts</code> | Évite les conflits de résolution et répond en grande partie à la question de licence ci-dessous                                       |
| <code>python=3.12</code> figé                           | Seule contrainte stricte : garantit le même interpréteur pour tous                                                                    |
| Autres paquets en <code>&gt;=</code>                    | Des versions figées au centième rendent la résolution fragile ; un échec de création bloquerait un participant avant même la séance 1 |

Pour ajouter un paquet : le déclarer dans `environment.yml` et relancer `installer.*`, la mise à jour est incrémentale. Ne jamais installer à la main sans passer par le fichier — le `--prune` le supprimerait au passage suivant.

**Attention. À faire avant l'envoi : testez `lancer_jupyterlab.bat` sur une machine vierge.**

Je n'ai pas pu exécuter conda pour valider la résolution des dépendances, ni lancer les scripts sous Windows. Vérifiez en particulier l'affichage des accents dans la fenêtre noire (les scripts utilisent `chcp 65001`) et le comportement de `installer.command` sous macOS, où Gatekeeper impose un premier clic droit → Ouvrir. Comptez 30 minutes.

Si vous voulez une uniformité stricte entre les postes, générez un fichier verrouillé après ce test : `conda env export > environment.lock.yml`, et distribuez celui-ci à la place.

### Attention. Point de licence à vérifier avant d'envoyer le guide

Anaconda indique sur sa page de téléchargement que **l'usage de ses offres dans une organisation de plus de 200 employés ou sous-traitants requiert une licence Business payante**, sauf éligibilité à un tarif réduit ou gratuit.

Si vos participants installent Anaconda sur des postes professionnels appartenant à une structure de cette taille, il faut trancher ce point **avant** de diffuser le guide. Trois options :

| Option                                              | Quand la choisir                                                                                                                                                            |
|-----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Vérifier si l'organisation détient déjà une licence | Beaucoup de grandes structures en ont une sans que les équipes le sachent. Premier réflexe : demander à la DSI.                                                             |
| Basculer sur <b>Miniforge</b>                       | Distribution communautaire (conda-forge), licence BSD, sans les conditions commerciales d'Anaconda. Installation un peu moins clé en main, mais aucune ambiguïté juridique. |
| Passer sur <b>Google Colab</b>                      | Rien à installer, rien à licencier. Solution la plus simple si les postes sont verrouillés de toute façon.                                                                  |

Un établissement d'enseignement bénéficie par ailleurs d'un accès académique gratuit via une adresse en `.edu`.

Je ne suis pas juriste et ces conditions évoluent : faites confirmer le point par votre service juridique ou votre DSI plutôt que de vous fier à ce document. Le sujet mérite d'être traité tôt, car il peut changer l'outil de la formation.

### Points de vigilance

**Prévoyez que 30 % des participants auront un problème.** C'est la norme, pas l'exception. Les causes fréquentes :

- Droits administrateur insuffisants sur les postes d'entreprise → anticiper avec la DSI **en amont**
- Antivirus bloquant l'installation
- Chemins contenant des accents ou des espaces → source de bugs difficiles à diagnostiquer plus tard
- Anaconda déjà installé en version ancienne

**Plan de secours obligatoire :** ayez un lien Google Colab prêt. Un participant bloqué en installation ne doit jamais rater la séance 1 — il travaille sur Colab ce jour-là, et on règle son installation à la pause.

Prévoyez également **30 minutes de marge en début de séance 1** pour les cas résiduels.

---

## 4. Module 1 — Fondamentaux du langage (séances 1 à 5)

Support Datacamp de référence : `Programming/Introduction to Python`, `Programming/Intermediate Python`

C'est le module le plus délicat. Il est aride, sans résultat visuel gratifiant, et c'est là que se produisent les abandons. Deux contre-mesures : chaque séance doit produire **un résultat tangible**, et chaque notion doit être introduite **par le problème qu'elle résout**, jamais par sa définition.

---

### Séance 1 — Premiers pas

**Objectif :** écrire et exécuter du code, manipuler variables et types.

| Notion                             | Points clés                                                                  |
|------------------------------------|------------------------------------------------------------------------------|
| L'interface Jupyter                | Cellules code/markdown, exécution (Maj+Entrée), noyau, redémarrage           |
| <code>print()</code>               | Premier retour visuel                                                        |
| Variables                          | Affectation, nommage, sensibilité à la casse                                 |
| Types de base                      | <code>int</code> , <code>float</code> , <code>str</code> , <code>bool</code> |
| Opérateurs                         | Arithmétiques, + sur les chaînes                                             |
| <code>type()</code> et conversions | <code>int()</code> , <code>str()</code> , <code>float()</code>               |
| Erreurs                            | Lire un message d'erreur — <b>notion critique</b>                            |

**Sur les erreurs :** consacrez-y 15 minutes pleines. Provoquez volontairement une `NameError`, une `TypeError`, une `SyntaxError`. Montrez comment lire la dernière ligne du message. Les débutants paniquent devant une erreur ; leur apprendre que c'est une information et non un échec est probablement le geste pédagogique le plus rentable de toute la formation.

**Atelier :** calculatrice de TVA, conversion d'unités, calcul d'un salaire annualisé à partir d'un mensuel.

**Travail personnel :** 8 micro-exercices de manipulation de variables.

## Séance 2 — Listes

**Objectif :** stocker et manipuler des collections.

| Notion             | Points clés                                                                              |
|--------------------|------------------------------------------------------------------------------------------|
| Création de listes | Homogènes puis hétérogènes                                                               |
| Indexation         | <b>Commence à 0</b> — insister lourdement                                                |
| Indices négatifs   | <code>[-1]</code> pour le dernier élément                                                |
| Slicing            | <code>[2:5]</code> , <code>[:3]</code> , <code>[3:]</code> , borne haute exclue          |
| Modification       | Affectation par indice, <code>append()</code> , <code>del</code> , <code>remove()</code> |
| Listes de listes   | Introduction à la structure tabulaire                                                    |
| Copie              | <code>list_b = list_a</code> ne copie pas ! Utiliser <code>.copy()</code>                |

**Erreur fondatrice à traiter :** l'indexation à 0 et la borne haute exclue du slicing génèrent des bugs pendant des semaines. Faites dessiner au tableau une liste avec ses indices. Faites-la manipuler physiquement (post-it numérotés) si le groupe est réceptif.

**Le piège de la copie** mérite un traitement explicite : `b = a` puis `b.append(x)` modifie `a` aussi. Démontrez-le, laissez le silence s'installer, puis expliquez. Ce moment de surprise ancre la notion durablement.

**Atelier :** gestion d'une liste de prix (ajout, suppression, extraction de sous-ensembles, calcul de total).

## Séance 3 — Fonctions et méthodes

**Objectif :** réutiliser du code, découvrir l'écosystème.

| Notion                            | Points clés                                                                                                                      |
|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Fonctions intégrées               | <code>len()</code> , <code>max()</code> , <code>min()</code> , <code>sum()</code> , <code>round()</code> , <code>sorted()</code> |
| <code>help()</code> et docstrings | <b>Apprendre à se documenter seul</b>                                                                                            |
| Méthodes                          | Différence objet/fonction : <code>ma_liste.append()</code> vs <code>len(ma_liste)</code>                                         |

| Notion              | Points clés                                                                                                                |
|---------------------|----------------------------------------------------------------------------------------------------------------------------|
| Méthodes de chaînes | <code>.upper()</code> , <code>.lower()</code> , <code>.replace()</code> , <code>.strip()</code> ,<br><code>.split()</code> |
| Méthodes de listes  | <code>.index()</code> , <code>.count()</code> , <code>.sort()</code>                                                       |
| Modules             | <code>import math</code> , notation pointée                                                                                |
| Alias d'import      | <code>import numpy as np</code> — préparer la suite                                                                        |

**Insistez sur `help()`.** Un participant qui sait consulter la documentation d'une fonction inconnue est autonome. Un participant qui attend qu'on lui donne la syntaxe ne le sera jamais. Faites un exercice où ils doivent utiliser une fonction que vous n'avez jamais présentée, en la découvrant seuls.

**Atelier :** nettoyage d'une liste de noms mal saisis (espaces, casse incohérente), statistiques descriptives manuelles sur une liste de nombres.

## Séance 4 — Conditions et dictionnaires

**Objectif :** faire des choix dans le code, structurer des données par clé.

| Notion                                                  | Points clés                                                                                                         |
|---------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Opérateurs de comparaison                               | <code>==</code> , <code>!=</code> , <code>&lt;</code> , <code>&gt;</code> , <code>&lt;=</code> , <code>&gt;=</code> |
| <code>==</code> vs <code>=</code>                       | Source d'erreur classique                                                                                           |
| Opérateurs booléens                                     | <code>and</code> , <code>or</code> , <code>not</code>                                                               |
| <code>if</code> / <code>elif</code> / <code>else</code> | <b>L'indentation fait la structure</b>                                                                              |
| Dictionnaires                                           | Création, accès par clé, ajout, suppression                                                                         |
| Parcours                                                | <code>.keys()</code> , <code>.values()</code> , <code>.items()</code>                                               |
| Quand liste vs dictionnaire                             | Critère de choix                                                                                                    |

**L'indentation** est spécifique à Python et déroute. Montrez visuellement qu'un bloc mal indenté change le sens du programme. Jupyter aide (indentation automatique) mais les participants collent parfois du code mal formaté.

**Atelier :** système de mention selon une note, répertoire téléphonique, catégorisation de clients selon leur chiffre d'affaires.

## Séance 5 — Boucles

**Objectif :** automatiser la répétition. **Séance charnière du module 1.**

| Notion                    | Points clés                                                 |
|---------------------------|-------------------------------------------------------------|
| Boucle <code>for</code>   | Sur une liste, sur une chaîne                               |
| <code>range()</code>      | Génération de séquences                                     |
| <code>enumerate()</code>  | Récupérer indice et valeur                                  |
| Boucle sur dictionnaire   | <code>.items()</code>                                       |
| Boucle <code>while</code> | Et son risque de boucle infinie                             |
| Accumulateur              | Pattern <code>total = 0</code> puis <code>total += x</code> |
| Compréhensions de listes  | <code>[x*2 for x in liste]</code> — introduction douce      |

Support complémentaire : [Programming/Python Toolbox](#) chapitre 2 pour les compréhensions.

Le **pattern accumulateur** est le schéma mental le plus important de la programmation impérative. Décomposez-le pas à pas au tableau, en écrivant l'état de chaque variable à chaque tour. Cette exécution manuelle est fastidieuse mais elle débloque durablement.

**Sur les compréhensions de listes :** introduisez-les comme une **écriture condensée** d'une boucle déjà comprise, jamais comme une notion nouvelle. Écrivez la boucle, puis la compréhension équivalente, côte à côte.

**Atelier :** calcul de statistiques sur une liste de ventes, filtrage conditionnel, comptage d'occurrences par catégorie.

\*\*\*\*Attention.\*\* Point de contrôle module 1.\*\* Proposez un mini-test d'auto-évaluation (non noté, 20 minutes) couvrant les séances 1 à 5. Il vous permet d'identifier les participants en difficulté **avant** d'attaquer pandas, où le retard devient irrattrapable. Prévoyez une séance de rattrapage optionnelle si plus de 3 participants décrochent.

---

## 5. Module 2 — NumPy (séances 6 et 7)

Support Datacamp : [Programming/Introduction to Python ch. 4, Data Manipulation/Introduction to NumPy](#)

Module court mais indispensable : pandas repose sur NumPy, et la logique vectorielle qu'on apprend ici est celle qu'on retrouvera partout ensuite.

---

### Séance 6 — Tableaux NumPy

**Objectif :** comprendre le calcul vectorisé.

| Notion                       | Points clés                                                          |
|------------------------------|----------------------------------------------------------------------|
| Le problème que résout NumPy | Multiplier une liste par 2 ne fonctionne pas comme attendu           |
| Création d'arrays            | Depuis une liste, <code>np.zeros()</code> , <code>np.arange()</code> |
| Homogénéité de type          | Contrainte fondamentale                                              |
| Opérations vectorisées       | <code>array * 2</code> , <code>array_a + array_b</code>              |
| Comparaison de performance   | Boucle vs vectorisation, chronométrée                                |
| Indexation et slicing        | Similaire aux listes                                                 |
| Filtrage booléen             | <code>array[array &gt; 10]</code> — <b>notion clé</b>                |
| Arrays 2D                    | <code>.shape</code> , indexation <code>[ligne, colonne]</code>       |

**Introduisez NumPy par la frustration.** Demandez-leur d'augmenter tous les prix d'une liste de 5 %. Ils écriront une boucle. Montrez `np.array(prix) * 1.05`. Le contraste fait le travail pédagogique.

**Le filtrage booléen** est la notion la plus importante de la séance : elle se retrouve identiquement dans pandas la semaine suivante. Consacrez-y au moins 30 minutes d'atelier.

**Lancement du projet fil rouge :** présentez le jeu de données en fin de séance. Laissez les participants l'explorer visuellement (dans un tableur, sans code). Faites-leur formuler par écrit 3 questions auxquelles ils aimeraient répondre. Ces questions guideront tout le reste de la formation et leur donneront une raison d'apprendre.



## Séance 7 — Statistiques avec NumPy

**Objectif :** calculer sur des tableaux, préparer l'analyse.

| Notion                  | Points clés                                                                                                                           |
|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Agrégations             | <code>.mean()</code> , <code>.median()</code> , <code>.std()</code> , <code>.sum()</code> , <code>.min()</code> , <code>.max()</code> |
| Agrégation par axe      | <code>axis=0</code> (colonnes) vs <code>axis=1</code> (lignes)                                                                        |
| <code>np.where()</code> | Conditionnel vectorisé                                                                                                                |
| Génération aléatoire    | <code>np.random.normal()</code> , <code>np.random.seed()</code>                                                                       |
| Reproductibilité        | Pourquoi fixer la graine aléatoire                                                                                                    |

L'argument **axis** est confondant pour tout le monde, y compris les praticiens confirmés. Schématisez au tableau : **axis=0** écrase les lignes, **axis=1** écrase les colonnes. Faites-leur vérifier systématiquement avec `.shape` avant et après.

**Atelier :** analyse d'un tableau de mesures multi-colonnes, détection de valeurs aberrantes par seuil.

## 6. Module 3 — pandas (séances 8 à 12)

Support Datacamp : Data Manipulation/Data Manipulation with pandas, Data Manipulation/Joining Data with pandas

**Cœur de la formation.** C'est l'outil qu'ils utiliseront quotidiennement. Cinq séances, c'est le minimum viable.

### Séance 8 — Découvrir les DataFrames

**Objectif :** charger et explorer un jeu de données.

| Notion                                                             | Points clés                                                                   |
|--------------------------------------------------------------------|-------------------------------------------------------------------------------|
| <code>pd.read_csv()</code>                                         | Chemin de fichier, encodage, séparateur                                       |
| Anatomie d'un DataFrame                                            | Lignes, colonnes, index                                                       |
| <code>.head()</code> , <code>.tail()</code>                        | Premier coup d'œil                                                            |
| <code>.info()</code>                                               | Types et valeurs manquantes                                                   |
| <code>.describe()</code>                                           | Statistiques descriptives                                                     |
| <code>.shape</code> , <code>.columns</code> , <code>.dtypes</code> | Métadonnées                                                                   |
| Series vs DataFrame                                                | Une colonne est une Series                                                    |
| Sélection de colonnes                                              | <code>df["col"]</code> vs <code>df[["col1", "col2"]]</code> — double crochets |

**Le rituel d'ouverture.** Enseignez une séquence systématique à exécuter sur tout nouveau jeu de données : `.head()` → `.info()` → `.describe()` → `.shape`. Faites-la répéter à chaque séance jusqu'à l'automatisme. C'est un réflexe professionnel qui les distinguera immédiatement.

**Problèmes d'encodage :** les fichiers français génèrent des caractères illisibles. Traitez `encoding="utf-8"` et `encoding="latin-1"` dès maintenant, ils y seront confrontés.

**Atelier :** exploration du jeu de données fil rouge avec le rituel d'ouverture.

## Séance 9 — Filtrer et trier

**Objectif :** extraire des sous-ensembles pertinents.

| Notion                                  | Points clés                                                               |
|-----------------------------------------|---------------------------------------------------------------------------|
| Tri                                     | <code>.sort_values()</code> , <code>ascending</code> , tri multi-colonnes |
| Filtrage simple                         | <code>df[df["prix"] &gt; 100]</code>                                      |
| Filtrage multiple                       | <code>&amp;</code> et <code> </code> avec <b>parenthèses obligatoires</b> |
| <code>.isin()</code>                    | Appartenance à une liste                                                  |
| <code>.loc</code> et <code>.iloc</code> | Sélection par label vs par position                                       |
| Colonnes calculées                      | <code>df["marge"] = df["px_vente"] - df["px_achat"]</code>                |
| Index                                   | <code>.set_index()</code> , <code>.reset_index()</code>                   |

**Les parenthèses dans les filtres multiples** sont une source d'erreur permanente. `df[df.a > 1 & df.b < 2]` échoue; `df[(df.a > 1) & (df.b < 2)]` fonctionne. Écrivez la règle au tableau et laissez-la affichée toute la séance.

`.loc` vs `.iloc` demande un traitement patient. Mnémotechnique : `iloc` = *integer location*.

**Atelier :** répondre aux 3 questions formulées en séance 6, uniquement par filtrage et tri.

## Séance 10 — Agréger et grouper

**Objectif :** synthétiser l'information. **Séance la plus rentable de la formation.**

| Notion                       | Points clés                                                                     |
|------------------------------|---------------------------------------------------------------------------------|
| Agrégations simples          | <code>.mean()</code> , <code>.sum()</code> , <code>.count()</code> sur colonnes |
| <code>.value_counts()</code> | Comptage de catégories, <code>normalize=True</code>                             |
| <code>.groupby()</code>      | Le concept central                                                              |
| Agrégations multiples        | <code>.agg(["mean", "std", "count"])</code>                                     |
| Groupeement multi-colonnes   | <code>groupby(["region", "produit"])</code>                                     |
| Tableaux croisés             | <code>.pivot_table()</code>                                                     |

**Le groupby est le moment « aha » de la formation.** Un participant qui maîtrise `groupby` peut répondre à 70 % des questions métier courantes. Prenez le temps.

Expliquez-le par la métaphore **split-apply-combine** : on découpe le jeu en paquets, on calcule sur chaque paquet, on recolle les résultats. Dessinez-le. Puis faites le parallèle avec les tableaux croisés dynamiques d'Excel — la plupart des participants connaissent, et ce pont vers leur savoir existant accélère énormément la compréhension.

**Atelier :** au moins 10 questions métier sur le jeu fil rouge, toutes résolues par `groupby`. C'est la séance où il faut le plus d'exercices.

## Séance 11 — Combiner des tables

**Objectif :** croiser plusieurs sources.

| Notion                      | Points clés                       |
|-----------------------------|-----------------------------------|
| Pourquoi plusieurs tables   | Notion de clé, modèle relationnel |
| <code>.merge()</code> inner | Jointure par défaut               |

| Notion                                                      | Points clés                                     |
|-------------------------------------------------------------|-------------------------------------------------|
| <code>left</code> , <code>right</code> , <code>outer</code> | Et leurs effets sur le nombre de lignes         |
| Clés de noms différents                                     | <code>left_on</code> , <code>right_on</code>    |
| <code>.concat()</code>                                      | Empilement vertical                             |
| Contrôle post-jointure                                      | <b>Vérifier <code>.shape</code> avant/après</b> |

**La jointure qui duplique les lignes** est le piège classique. Un `merge` sur une clé non unique multiplie silencieusement les lignes et fausse tous les totaux ensuite. Enseignez le réflexe : vérifier le nombre de lignes après chaque jointure, et utiliser `validate="one_to_one"` ou `validate="one_to_many"` pour se protéger.

Schématisez les 4 types de jointure avec des diagrammes de Venn au tableau.

**Atelier** : enrichir le jeu fil rouge avec une table complémentaire (référentiel produits, table géographique, calendrier).

## Séance 12 — Consolidation pandas

**Objectif** : fluidifier, sans notion nouvelle.

Séance entièrement consacrée à la pratique. **Ne présentez rien de neuf**. Le module pandas est dense ; sans temps de consolidation, les acquis restent fragiles.

**Déroulé suggéré** :

- 30 min — Reprise collective des difficultés remontées, correction des erreurs fréquentes
- 60 min — Atelier long : une analyse complète guidée sur un jeu de données **nouveau**
- 45 min — Défi en binômes : 8 questions métier chronométrées, correction collective
- 15 min — Bilan, constitution d'un aide-mémoire personnel

**L'aide-mémoire** est un livrable à valoriser : faites-leur écrire leur propre fiche de synthèse pandas, une page, avec les commandes qu'*eux* trouvent utiles. La rédiger vaut mieux que la recevoir.

## 7. Module 4 — Import et nettoyage (séances 13 et 14)

Support Datacamp : Data Preparation/Introduction to Importing Data in Python, Data Preparation/Cleaning Data in Python

Module souvent négligé dans les formations, alors qu'il représente **60 à 80 % du temps réel** d'un analyste. C'est aussi ce qui sépare l'exercice scolaire du travail professionnel.

### Séance 13 — Importer depuis toutes les sources

**Objectif** : ne jamais être bloqué par un format de fichier.

| Notion                | Points clés                                                                                                      |
|-----------------------|------------------------------------------------------------------------------------------------------------------|
| CSV avancé            | <code>sep</code> , <code>decimal</code> , <code>encoding</code> , <code>skiprows</code> , <code>na_values</code> |
| Le CSV français       | Séparateur ; et décimale , — <b>cas très fréquent</b>                                                            |
| Excel                 | <code>pd.read_excel()</code> , <code>sheet_name</code> , feuilles multiples                                      |
| Sélection de colonnes | <code>usecols</code> pour les gros fichiers                                                                      |
| Bases SQL             | <code>sqlite3</code> / <code>SQLAlchemy</code> , <code>pd.read_sql()</code>                                      |
| Export                | <code>.to_csv()</code> , <code>.to_excel()</code> , <code>index=False</code>                                     |

**Le CSV français mérite un traitement dédié :** `pd.read_csv(f, sep=";", decimal=".", encoding="latin-1")`. Vos participants rencontreront ce cas dès leur première utilisation professionnelle.

**Atelier :** importer 5 fichiers volontairement problématiques (en-têtes décalés, encodage exotique, séparateur inhabituel, lignes de commentaire en tête, valeurs manquantes codées N/A, -, 999).

---

## Séance 14 — Nettoyer des données réelles

**Objectif :** transformer des données sales en données exploitables.

| Notion                  | Points clés                                                                          |
|-------------------------|--------------------------------------------------------------------------------------|
| Diagnostic              | <code>.info()</code> , <code>.isna().sum()</code> , <code>.duplicated().sum()</code> |
| Types incorrects        | <code>.astype()</code> , <code>pd.to_numeric(errors="coerce")</code>                 |
| Dates                   | <code>pd.to_datetime()</code> , <code>format</code> , accesseur <code>.dt</code>     |
| Doublons                | <code>.drop_duplicates()</code> , <code>subset</code> , <code>keep</code>            |
| Valeurs manquantes      | <code>.dropna()</code> , <code>.fillna()</code> — <b>et la question du choix</b>     |
| Chaînes incohérentes    | <code>.str.strip()</code> , <code>.str.lower()</code> , <code>.str.replace()</code>  |
| Catégories incohérentes | <code>.replace()</code> avec dictionnaire de correspondance                          |
| Aberrations             | Détection par seuil, par écart-type                                                  |

**Sur les valeurs manquantes**, il faut aller au-delà de la technique. La vraie question n'est pas *comment* les traiter mais *pourquoi elles manquent*. Une valeur absente parce que non applicable, non collectée, ou perdue, appelle trois traitements différents. Supprimer par défaut biaise l'analyse ; remplacer par la moyenne aussi.

Faites-en un moment de discussion collective sur un cas concret. Ce type de réflexion est exactement ce qui distingue un analyste d'un exécutant.

**Atelier :** nettoyage complet d'un jeu de données volontairement dégradé, avec journal des décisions prises et de leur justification.

---

## 8. Module 5 — Visualisation (séances 15 à 17)

Support Datacamp : Data Visualization/Introduction to Data Visualization with Matplotlib, .../with Seaborn

---

### Séance 15 — Matplotlib

**Objectif :** produire les graphiques de base.

| Notion                | Points clés                                         |
|-----------------------|-----------------------------------------------------|
| Structure figure/axes | <code>fig, ax = plt.subplots()</code>               |
| Courbe                | <code>ax.plot()</code>                              |
| Nuage de points       | <code>ax.scatter()</code>                           |
| Barres                | <code>ax.bar()</code>                               |
| Histogramme           | <code>ax.hist()</code> , choix du nombre de classes |
| Habillage             | Titre, labels d'axes, légende                       |
| Sous-graphiques       | <code>plt.subplots(2, 2)</code>                     |
| Export                | <code>plt.savefig()</code> , <code>dpi</code>       |

**Adoptez d'emblée la syntaxe orientée objet** (`fig, ax = plt.subplots()`). La syntaxe `plt.plot()` est plus courte mais devient ingérable dès qu'on veut plusieurs graphiques. Autant prendre la bonne habitude immédiatement.

**Un graphique sans titre ni labels d'axes est un graphique incomplet.** Posez la règle dès la première minute et refusez systématiquement les rendus non habillés. C'est une exigence facile à tenir et qui construit un réflexe professionnel durable.

---

## Séance 16 — Seaborn

**Objectif :** produire rapidement des graphiques statistiques élégants.

| Notion                                           | Points clés                                                    |
|--------------------------------------------------|----------------------------------------------------------------|
| Positionnement                                   | Seaborn au-dessus de Matplotlib                                |
| Format « tidy »                                  | Une observation par ligne, une variable par colonne            |
| <code>scatterplot</code> , <code>lineplot</code> | Avec <code>hue</code> , <code>size</code> , <code>style</code> |
| <code>countplot</code> , <code>barplot</code>    | Variables catégorielles                                        |
| <code>boxplot</code> , <code>violinplot</code>   | Distributions comparées                                        |
| <code>histplot</code> , <code>kdeplot</code>     | Distributions univariées                                       |
| <code>heatmap</code>                             | Matrice de corrélation                                         |
| <code>pairplot</code>                            | Exploration multivariée rapide                                 |
| Style                                            | <code>sns.set_style()</code> , palettes de couleurs            |

**Le paramètre `hue`** est le point fort de Seaborn : ajouter une dimension d'analyse en un mot-clé. Démontrez-le sur le jeu fil rouge, l'effet est spectaculaire.

**Atelier :** produire 6 graphiques différents sur le jeu fil rouge, chacun répondant à une question précise formulée à l'avance.

---

## Séance 17 — Communiquer par le graphique

**Objectif :** passer du graphique correct au graphique convaincant.

| Notion                     | Points clés                                                                |
|----------------------------|----------------------------------------------------------------------------|
| Choisir le bon type        | Comparaison / évolution / distribution / relation / composition            |
| Erreurs classiques         | Axe Y tronqué, camembert à 12 parts, 3D inutile                            |
| Couleur                    | Usage fonctionnel, palettes séquentielles vs catégorielles                 |
| Accessibilité              | Daltonisme, lisibilité en noir et blanc                                    |
| Densité d'information      | Supprimer ce qui n'informe pas                                             |
| Titre porteur de message   | « Les ventes ont chuté de 12 % au T3 » plutôt que « Ventes par trimestre » |
| Graphiques pour un rapport | Format, résolution, cohérence visuelle                                     |

**Le titre porteur de message** est un levier simple et très efficace : un titre qui énonce la conclusion plutôt que le contenu transforme un graphique descriptif en argument. Faites-leur réécrire les titres de leurs graphiques de la séance 16.

**Exercice de critique :** présentez 5 graphiques défectueux (issus de la presse ou de rapports réels), faites-en identifier les défauts collectivement, puis faites-les corriger en code. L'analyse critique développe le jugement bien plus vite que la production.

---

## 9. Module 6 — Statistiques et EDA (séances 18 et 19)

Support Datacamp : Probability & Statistics/Introduction to Statistics in Python,  
Exploratory Data Analysis/Exploratory Data Analysis in Python

---

### Séance 18 — Statistiques descriptives

**Objectif :** décrire rigoureusement une distribution.

| Notion                                         | Points clés                                          |
|------------------------------------------------|------------------------------------------------------|
| Tendance centrale                              | Moyenne, médiane, mode                               |
| Moyenne vs médiane                             | <b>Quand la moyenne trompe</b>                       |
| Dispersion                                     | Étendue, variance, écart-type                        |
| Quantiles                                      | Quartiles, déciles, lecture d'une boîte à moustaches |
| Forme                                          | Asymétrie, distributions bimodales                   |
| Loi normale                                    | Repères 68/95/99,7                                   |
| Corrélation                                    | Coefficient, matrice, interprétation                 |
| <b>Corrélation <math>\neq</math> causalité</b> | Traitement approfondi                                |

**Moyenne vs médiane :** l'exemple des salaires est imbattable. Une entreprise de 10 personnes où 9 gagnent 2 000 € et le dirigeant 50 000 € a un salaire moyen de 6 800 € — chiffre qui ne décrit personne. Cet exemple s'ancre durablement et fait comprendre en une minute ce qu'un cours théorique met une heure à transmettre.

**Corrélation et causalité :** consacrez-y 20 minutes. Utilisez des corrélations absurdes documentées, faites chercher les variables cachées. C'est une compétence de jugement plus qu'une compétence technique, et c'est précisément ce qu'on attend d'un analyste.

---

### Séance 19 — Démarche exploratoire complète

**Objectif :** structurer une analyse de bout en bout.

| Étape             | Contenu                                          |
|-------------------|--------------------------------------------------|
| 1. Question       | Formuler précisément ce qu'on cherche            |
| 2. Reconnaissance | Le rituel d'ouverture, comprendre chaque colonne |
| 3. Qualité        | Diagnostic manquants / doublons / aberrations    |
| 4. Univarié       | Chaque variable séparément                       |
| 5. Bivarié        | Croisements pertinents                           |
| 6. Hypothèses     | Formuler, tester, réfuter                        |
| 7. Synthèse       | 3 à 5 conclusions étayées                        |

**Le point le plus important de cette séance :** apprendre à **s'arrêter**. Les débutants explorent sans fin, produisent 40 graphiques et aucune conclusion. Imposez une contrainte stricte — maximum 6 graphiques, exactement 3 conclusions. La contrainte force la hiérarchisation, qui est le vrai travail analytique.

**Enseignez aussi le résultat négatif.** « Nous n'avons pas trouvé de relation entre X et Y » est une conclusion valide et utile. Les débutants ont tendance à forcer l'interprétation pour avoir « quelque chose à dire ».

**Atelier :** démarche complète en binômes sur un jeu de données nouveau, en 90 minutes chronométrées.

---

## 10. Séance 20 — Restitution du projet final

Format :

| Temps       | Activité                                               |
|-------------|--------------------------------------------------------|
| 0-20 min    | Derniers ajustements, aide technique                   |
| 20-110 min  | Présentations : 10 min par binôme + 5 min de questions |
| 110-130 min | Retours collectifs, points forts et axes de progrès    |
| 130-150 min | Feuille de route post-formation, clôture               |

**Consignes du projet** (à distribuer en séance 12, soit 8 semaines de travail) :

- Jeu de données au choix, validé par le formateur
- Notebook propre, commenté, exécutable de bout en bout
- Structure imposée : contexte → question → données → nettoyage → analyse → conclusions → limites
- 3 à 6 graphiques, tous justifiés
- 3 conclusions étayées par les données
- **Une section « limites de l'analyse » obligatoire**

**La section « limites » est le meilleur révélateur de maturité analytique** dont vous disposez pour l'évaluation. Un participant capable d'identifier les faiblesses de sa propre analyse a compris l'essentiel du métier. Pondez-la fortement.

### Grille d'évaluation

| Critère                                            | Poids |
|----------------------------------------------------|-------|
| Code fonctionnel et lisible                        | 20 %  |
| Pertinence du nettoyage et justification des choix | 20 %  |
| Qualité et pertinence des visualisations           | 20 %  |
| Solidité des conclusions                           | 20 %  |
| Lucidité sur les limites                           | 10 %  |
| Clarté de la présentation orale                    | 10 %  |

### Feuille de route post-formation

Concluez en indiquant les directions possibles, avec les supports Datacamp correspondants déjà présents dans votre dossier :

| Direction                   | Cours disponibles dans le dossier                                       |
|-----------------------------|-------------------------------------------------------------------------|
| Approfondir pandas          | Reshaping Data with pandas, Writing Efficient Code with pandas          |
| Statistiques inférentielles | Hypothesis Testing in Python, Sampling in Python                        |
| Machine learning            | Supervised Learning with scikit-learn, Unsupervised Learning in Python  |
| Programmation avancée       | Writing Functions in Python, Object-Oriented Programming in Python      |
| Séries temporelles          | Manipulating Time Series Data in Python, Time Series Analysis in Python |
| Automatisation              | Introduction to APIs in Python, Web Scraping in Python                  |
| Tableaux de bord            | Building Dashboards with Dash and Plotly                                |

**Message de clôture à porter :** ils ne sont pas experts, et c'est normal. Ils sont autonomes — capables d'apprendre seuls la suite. C'est exactement ce qu'une formation d'initiation doit produire.

---

## 11. Fil transversal — Code répliquable et dossier de travail organisé

### 11.1 Le principe : la convention d'abord, l'explication ensuite

La reproductibilité ne peut pas être un module. Un participant qui ne sait pas ce qu'est une variable ne peut pas comprendre pourquoi un chemin absolu pose problème — la notion n'a aucune prise sur son expérience. Mais si on attend qu'il soit prêt à comprendre, les mauvaises habitudes sont déjà installées, et les défaire coûte trois fois plus cher que de les prévenir.

La solution est de dissocier les deux temps :

**On impose la structure dès le premier jour, sans l'expliquer. On explique le pourquoi plus tard, au moment précis où le participant vient d'en ressentir le besoin.**

Concrètement : le dossier de travail est créé **avant même la séance 1** (il figure à l'étape 4 du guide d'installation). Personne ne demande pourquoi, parce que ça arrive avec l'installation, comme une évidence. Puis en séance 8, quand un participant n'arrive pas à ouvrir un fichier qui fonctionnait chez son voisin, l'explication tombe sur un terrain préparé.

### 11.2 Calendrier d'introduction

| Séance          | Ce qu'on introduit                                                                                  | Temps   | Statut                 |
|-----------------|-----------------------------------------------------------------------------------------------------|---------|------------------------|
| <b>Avant S1</b> | L'environnement<br><b>formation-python</b> ,<br>l'arborescence de travail,<br>les règles de nommage | —       | Imposé, non expliqué   |
| <b>S1</b>       | L'en-tête de notebook, la<br>convention de nommage<br>des fichiers                                  | 10 min  | Imposé, non expliqué   |
| <b>S5</b>       | Commenter son code,<br>nommer ses variables                                                         | 15 min  | Première justification |
| <b>S8</b>       | <b>Chemins relatifs — la<br/>leçon centrale</b>                                                     | 25 min  | Explication complète   |
| <b>S12</b>      | « Redémarrer et tout<br>exécuter », structure<br>narrative du notebook                              | 30 min  | Explication complète   |
| <b>S14</b>      | Le journal des décisions<br>de nettoyage                                                            | 20 min  | Explication complète   |
| <b>S19</b>      | Intégration : la démarche<br>complète et<br>reproductible                                           | Intégré | Mise en pratique       |
| <b>S20</b>      | Critère d'évaluation du<br>projet                                                                   | —       | Évalué                 |

### 11.3 Le détail, séance par séance

**Avant la séance 1 — l'environnement et l'arborescence.** Les deux sont mis en place pendant l'installation, dans le même mouvement et avec la même logique : on applique, on n'explique pas.

L'environnement conda mérite une précision, car il peut sembler contredire la section 11.4 ci-dessous. **Utiliser un environnement n'est pas enseigner les environnements.** Le participant double-clique sur un fichier,



attend, puis double-clique sur un autre à chaque séance. Il ne tape aucune commande et ne voit jamais le mot « conda ». C'est un geste, pas un concept. Le geste s'installe sans coût cognitif ; le concept, lui, viendra en séance 12 ou pas du tout.

L'intérêt pédagogique est réel : c'est la démonstration matérielle que *tout le monde travaille dans le même environnement*. Quand vous poserez en séance 8 la règle « un notebook doit fonctionner sur n'importe quel ordinateur », vous pourrez y renvoyer — la condition est déjà remplie, ils l'ont installée eux-mêmes.

Le geste à ancrer dès la première séance : **on ouvre toujours JupyterLab par lancer\_jupyterlab, jamais par Anaconda Navigator**. Le lanceur active l'environnement avant de démarrer JupyterLab ; passer par Navigator court-circuite cette activation et produit des erreurs incompréhensibles pour un débutant — noyau qui meurt sans message, ou `ModuleNotFoundError` alors que « tout était installé ». Dites-le une fois en séance 1, et redites-le à chaque fois que vous voyez quelqu'un ouvrir Navigator.

**L'arborescence** est créée par l'installateur, pas par le participant : `seances/`, `donnees/` (avec `brut/` et `propre/`), `resultats/`, `projet_final/`. La recevoir toute faite vaut mieux que la créer soi-même — cela supprime les **données** avec accent, les **Séances** avec majuscule, et les dossiers oubliés. Restent trois règles de nommage à respecter pour leurs propres fichiers : pas d'accent, pas d'espace, dates en AAAA-MM-JJ. La règle la plus importante à cet instant est celle du dossier `brut/` : **les données reçues ne sont jamais modifiées**. C'est intuitif, ça ne demande aucune connaissance technique, et c'est le fondement de tout le reste.

**Séance 1 — l'en-tête de notebook**. Dix minutes, présentées comme « voici comment on travaille ici », sans théorie. Chaque notebook commence par une cellule Markdown contenant quatre lignes : titre, date, auteur, objectif en une phrase. Et chaque notebook est nommé `MN_sujet.ipynb` — `01_variables.ipynb`, `02_listes.ipynb`. Le numéro à deux chiffres garantit le tri correct.

L'effet recherché est celui d'une norme d'atelier, pas d'une leçon. On ne justifie rien, on montre et on applique.

**Séance 5 — commenter et nommer**. Après les boucles, leur code devient assez long pour qu'ils s'y perdent eux-mêmes. C'est le premier moment où une justification porte. Le déclencheur pédagogique : faites-leur relire un code qu'ils ont écrit à la séance 2. Beaucoup ne le comprendront plus. La leçon s'écrit toute seule.

Deux règles suffisent : des noms de variables qui décrivent le contenu (`prix_moyen` et non `x`), et un commentaire qui explique **pourquoi** et non **quoi**. `# on exclut les commandes annulées` est utile ; `# on filtre le dataframe` est du bruit.

**Séance 8 — les chemins relatifs**. C'est la **leçon centrale**. Elle arrive au premier `pd.read_csv()`, c'est-à-dire au premier instant où le code touche le monde extérieur au notebook.

Le dispositif : demandez-leur d'ouvrir un fichier de données, sans consigne particulière. Presque tous écriront quelque chose comme `pd.read_csv("C:/Users/Fiacre/Documents/formation_python/donnees/brut/ventes.csv")`. Faites-leur alors échanger leur notebook avec leur voisin et l'exécuter. **Aucun ne fonctionne**. Le constat est immédiat, collectif, et impossible à oublier.

Vous introduisez ensuite le chemin relatif — `pd.read_csv("../donnees/brut/ventes.csv")` — et la règle générale : *un notebook doit fonctionner sur n'importe quel ordinateur où le dossier de travail a été copié*. C'est la définition opérationnelle de la reproductibilité, formulée en une phrase qu'ils viennent de comprendre par l'échec.

Prévoyez 25 minutes : la notation `..` pour remonter d'un niveau demande un schéma au tableau et un peu de pratique.

**Séance 12 — le test « Redémarrer et tout exécuter »**. La consolidation pandas est le bon moment : ils ont maintenant des notebooks longs, où ils ont exécuté les cellules dans le désordre, effacé des lignes, réécrit des morceaux.

Le dispositif, aussi brutal que le précédent : faites-leur exécuter **Kernel → Restart Kernel and Run All Cells** sur leur notebook de la séance 10. Une bonne moitié plantera. La raison est contre-intuitive et c'est précisément ce qui la rend mémorable : **un notebook qui affiche des résultats corrects n'est pas nécessairement un notebook qui fonctionne**. L'ordre d'exécution réel diffère de l'ordre de lecture.

Posez alors la règle, et tenez-la jusqu'à la fin de la formation :

## Un notebook qui ne passe pas le test « Restart & Run All » n'est pas terminé.

Ajoutez la structure narrative : un notebook se lit comme un document, pas comme un brouillon. Titre, sections en Markdown, une cellule par idée, les imports regroupés en haut, et le code mort supprimé. Faites-leur restructurer un de leurs anciens notebooks — le contraste avant/après est parlant.

**Séance 14 — le journal des décisions.** Le nettoyage est fait de choix : supprimer ou remplacer les valeurs manquantes, exclure ou conserver les valeurs extrêmes, uniformiser telle catégorie. Ces choix modifient les conclusions, et un mois plus tard leur auteur ne s'en souvient plus.

La consigne est simple : chaque décision de nettoyage est précédée d'une cellule Markdown indiquant **ce qui a été fait, pourquoi, et ce qui a été écarté**. Deux lignes suffisent.

C'est aussi le moment de faire le lien avec la règle du dossier **brut/** posée avant la séance 1 : les données d'origine sont intactes, les transformations sont dans le code, donc **toute décision est révisable**. Les participants découvrent que la contrainte qu'on leur a imposée cinq mois plus tôt les protège. C'est la boucle qui se referme, et elle est très efficace pédagogiquement.

**Séances 19 et 20 — intégration et évaluation.** Plus rien de nouveau : la démarche exploratoire complète mobilise tout ce qui précède. Le projet final est soumis au test « Restart & Run All » en votre présence, et le critère « code fonctionnel et lisible » de la grille d'évaluation en dépend directement.

### 11.4 Ce qu'il ne faut surtout pas enseigner

La tentation est réelle d'aller plus loin — elle serait contre-productive. Sur 50 heures et avec des débutants complets, **Git, les scripts .py, le versionnage des dépendances et les tests automatisés n'ont pas leur place**. Ce sont des outils dont la valeur n'apparaît que sur des projets collaboratifs et durables. Introduits trop tôt, ils produisent de la confusion et de l'abandon, sans bénéfice.

La même réserve vaut pour **la théorie des environnements conda**. Les participants en utilisent un, mais n'ont aucun besoin de savoir créer, cloner ou exporter un environnement, ni de comprendre la résolution de dépendances. Si la question surgit — elle surgira, quelqu'un demandera toujours « c'est quoi cette fenêtre noire ? » — répondez en une phrase : « *c'est une boîte qui contient les outils de la formation, pour que tout le monde ait exactement les mêmes* », et poursuivez. Une explication complète à ce stade coûte vingt minutes et ne laisse aucune trace.

La seule concession raisonnable : en séance 20, mentionner leur existence en deux minutes, comme la suite naturelle du chemin, et les inscrire dans la feuille de route post-formation. Rien de plus.

### 11.5 Le point qui décide de tout : votre propre exemple

Aucune de ces règles ne tiendra si vos notebooks de démonstration ne les respectent pas. Les participants copient ce qu'ils voient à l'écran, pas ce qu'ils entendent.

Cela implique une discipline sans exception de votre part, dès la séance 1 :

- Vos notebooks portent la même arborescence et les mêmes conventions de nommage que les leurs
- Vous n'utilisez **jamais** un chemin absolu en démonstration, même en séance 1, même « pour aller vite »
- Vos notebooks commencent par l'en-tête que vous exigez d'eux
- Vous exécutez « Restart & Run All » devant eux avant chaque démonstration à partir de la séance 12

Un formateur qui tape un chemin absolu en séance 6 « parce que c'est plus rapide » détruit en dix secondes ce qu'il tentera d'installer en séance 8.

### 11.6 Le rituel de clôture

Cinq minutes en fin de chaque séance, à partir de la séance 8 :

1. Le notebook est nommé selon la convention

2. Il est enregistré au bon endroit
3. Il porte son en-tête
4. Il passe « Restart & Run All » (*à partir de S12*)

Cinq minutes hebdomadaires sur vingt séances représentent moins de 2 % du temps de formation. C'est le meilleur rapport coût/bénéfice de tout le programme : la rigueur méthodologique est ce qui reste le plus longtemps, bien après que la syntaxe exacte de `pivot_table` a été oubliée.

---

## 12. Recommandations opérationnelles

### 12.1 Avant le démarrage

- ☐ Valider l'accès administrateur des postes avec la DSI
- ☐ Envoyer le guide d'installation 10 jours avant
- ☐ Préparer le lien Colab de secours
- ☐ Sélectionner et préparer le jeu de données fil rouge
- ☐ Constituer une bibliothèque de jeux de données pour les ateliers
- ☐ Vérifier la disposition de la salle : les participants doivent voir l'écran **et** leur clavier

### 12.2 Gérer l'hétérogénéité

Elle est inévitable et s'aggrave à chaque séance. Trois leviers :

**Exercices à deux niveaux.** Chaque atelier comporte un socle obligatoire et 2 ou 3 questions bonus. Les rapides ne s'ennuient pas, les autres ne se sentent pas en échec.

**Binômes tournants.** Associez un participant à l'aise avec un participant en difficulté, en changeant les paires chaque séance. Expliquer consolide chez celui qui explique ; c'est un bénéfice mutuel, pas un sacrifice.

**Permanence optionnelle.** 30 minutes avant ou après la séance, pour ceux qui le souhaitent. Suffisant pour éviter la plupart des décrochages.

### 12.3 Le risque principal : l'abandon en cours de route

Sur un format hebdomadaire de 5 mois, l'attrition est le risque numéro un — bien avant la difficulté technique. Les leviers qui fonctionnent :

- **Une victoire visible dès la séance 1.** Personne ne doit repartir sans avoir fait fonctionner quelque chose.
- **Le projet fil rouge sur des données qui les concernent.** L'engagement métier est le meilleur antidote à l'abandon.
- **Un canal d'entraide permanent** (Teams, Slack, WhatsApp) où les questions entre séances trouvent réponse.
- **Rattraper immédiatement les absences.** Un participant qui manque une séance de pandas sans rattrapage est perdu. Prévoyez un notebook de rattrapage autonome pour chaque séance.

### 12.4 Le rythme hebdomadaire : sa faiblesse à compenser

Une semaine entre deux séances, c'est assez long pour oublier. Contre-mesures :

- **Le travail personnel n'est pas optionnel.** 1h minimum entre chaque séance, et il est corrigé collectivement en ouverture.
- **Les 15 premières minutes de chaque séance sont un rappel actif** — pas une récapitulation par le formateur, mais des questions posées aux participants.
- **Un aide-mémoire cumulatif** enrichi par les participants eux-mêmes à chaque séance.

## 12.5 Sur les supports Datacamp

Vous disposez de 641 PDF, dont environ 25 sont directement utiles à cette formation. Deux mises en garde :

Ces slides sont **en anglais** et conçus pour de la vidéo — ils fonctionnent mal en projection frontale (peu de texte, dépendants du commentaire oral). Utilisez-les comme **source de contenu et de séquençement**, pas comme support de projection tel quel. Prévoyez de reconstruire vos propres slides en français.

Le **niveau de langue** peut être un obstacle selon votre public. Si les participants ne sont pas à l'aise en anglais, il faudra traduire les concepts clés et fournir un glossaire anglais-français des termes techniques — les noms de fonctions restant en anglais par nature.

---

## 13. Récapitulatif du volume horaire

| Module                              | Séances   | Heures      |
|-------------------------------------|-----------|-------------|
| Fondamentaux du langage             | 5         | 12h30       |
| NumPy                               | 2         | 5h          |
| pandas                              | 5         | 12h30       |
| Import et nettoyage                 | 2         | 5h          |
| Visualisation                       | 3         | 7h30        |
| Statistiques et EDA                 | 2         | 5h          |
| Restitution                         | 1         | 2h30        |
| <b>Total présentiel</b>             | <b>20</b> | <b>50h</b>  |
| Travail personnel                   | —         | ~25h        |
| Projet final                        | —         | ~15h        |
| <b>Total engagement participant</b> |           | <b>~90h</b> |

---

*Programme construit à partir des supports Datacamp disponibles dans le dossier de formation.*